Penetration Test Report - "Lo-Fi" Machine

Field	Detail
Client	TryHackMe
Target IP	10.65.154.115
Test Date	March 15, 2026
Consultant	Bender2997_

1. Executive Summary

The objective of this assessment was the IP address 10.65.154.115, identified as the "Lo-Fi" machine, to validate the existence of compromise vectors. A critical Local File Inclusion (LFI) vulnerability was identified and exploited on the web server. The system was configured with a basic blocklist that was bypassed using a *path traversal* technique with the Gobuster tool and a specific *fuzzing* dictionary. This process allowed access to sensitive operating system files. The criticality of this finding is classified as Critical, as a successful LFI exploitation can lead to the reading of confidential system information, and even remote code execution under certain configurations.

2. Methodology and Scope

Field	Objective	Detail
Environment	"Lo-Fi" Web Server	Target IP 10.65.154.115
Scope	Black-Box Testing	Limited to the identification and exploitation of a single critical vulnerability.
Methodology		An iterative approach was followed: port reconnaissance, initial LFI testing, and advanced *fuzzing* with Gobuster to bypass security filters.

3. Reconnaissance and Discovery Phase

An exhaustive TCP port scan was performed on the target IP to identify the attack surface.

Nmap Command Used:

nmap -T5 -sS -sCV -Pn -n -p- --open 10.65.154.115

Scan Results:

Port	State	Service	Version
22/tcp	Open	ssh	OpenSSH 8.2p1 Ubuntu 4ubuntu0.4 (protocol 2.0)
80/tcp	Open	http	Apache httpd 2.2.22 ((Ubuntu))

Service Analysis:

The web server on port 80 was running **Apache httpd 2.2.22** and its page title was **"Lo-Fi Music"**. The existence of the Local File Inclusion (LFI) vulnerability was tested by injecting paths to access the /etc/passwd file.

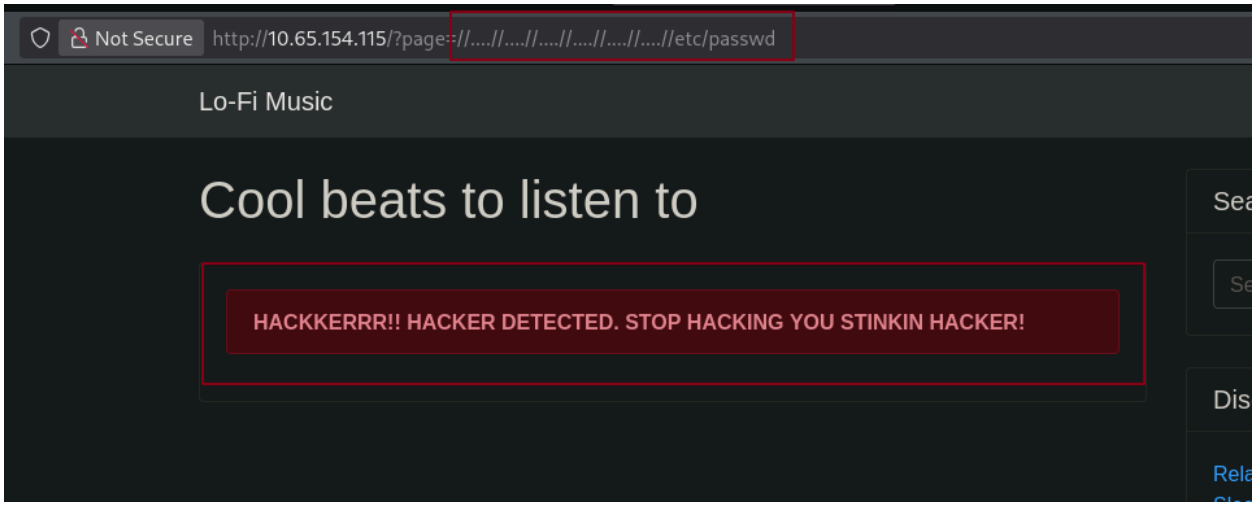
4. Finding: Local File Inclusion (LFI) - Filter Evasion

ID	LVF-001
Severity	Critical (High)
CWE	CWE-22 (Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal'))

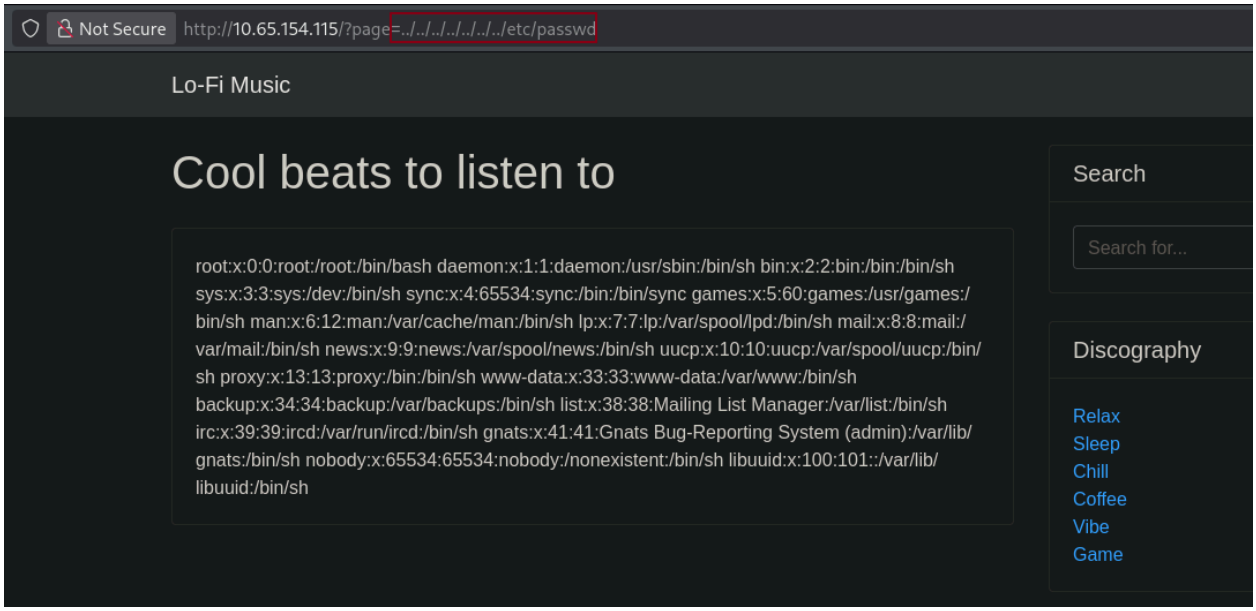
Vulnerability Description:

The web application parameter susceptible to LFI initially blocked *path traversal* sequences (../). An initial LFI attempt with excessive paths was detected and blocked, returning the message:

"HACKKERRR!! HACKER DETECTED. STOP HACKING YOU STINKIN HACKER!".



However, path *fuzzing* with the Gobuster tool allowed the discovery of a valid *path traversal* sequence that evaded the security filter.



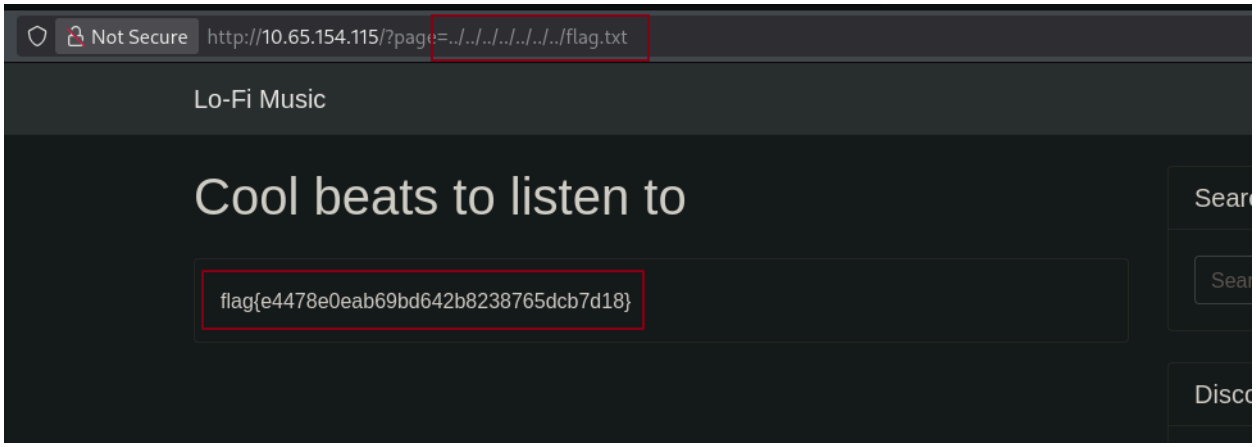
Impact:

Successful exploitation of the LFI vulnerability allows an unauthenticated attacker to read sensitive operating system files, such as /etc/passwd, exposing user account information. This access to sensitive information represents a direct risk to system confidentiality and can be an initial step for privilege escalation or obtaining remote access.

Proof of Concept (PoC):

1. **Initial Failed Attempt (Blocked):**
   http://10.65.154.115/?page=//....//....//....//....//....//....//etc/passwd
2. **Fuzzing for Filter Evasion:**
   Gobuster was used with a *path traversal* dictionary to find the correct value:
   gobuster dir -u 10.65.154.115 -w /usr/share/wordlists/seclists/Fuzzing/LFI/LFI-LFISuite-pathtotest-huge.txt -t 100 -n --hl 305
3. **Correct *Path Traversal* Value:**
   The tool identified the valid path: ../../../../../../../etc/passwd
4. **Successful Access and *Flag* Retrieval:**
   Exploitation with the correct path led to the validation of system compromise.

Flag Obtained (Compromise Indicator):
Flag{e4478e0eab69bd642b8238765dcb7d18}



5. Conclusions and Mitigation Recommendations

Conclusion:
The penetration test demonstrated a critical failure in user input validation on the web application parameter. Although a defense mechanism (character blocklist) existed, it

was ineffective against advanced *fuzzing* techniques, which allowed Local File Inclusion and system compromise.

Critical Recommendations (LVF-001):

1. **Principle of Least Privilege (Critical Priority):**
    - **Eliminate LFI/Path Traversal functionality:** The application must be redesigned to avoid using user input to directly include files.
    - **Whitelist vs. Blacklist:** Implement a strict **Whitelist** mechanism that only allows a defined set of secure file names and/or directories. Never rely solely on a blacklist to filter malicious sequences.
2. **Robust Input Validation (High Priority):**
    - Ensure that all user inputs are strictly validated. This includes sanitizing the input, eliminating path sequence characters (. and /) before the value is used in any file or path operation.
3. **System Hardening (Medium Priority):**
    - Ensure that the web application user (e.g., www-data or apache) has the fewest possible privileges and is confined to its own directory (chroot or containers) to limit the scope of any exploitation.